TPS Manipuler les fonctionnalités importantes de Redux

Partie 1 : Panier de Fruits

Exercice 1

Créer AjoutFruit.jsx avec un input + bouton Ajouter → dispatch dans Redux.

Exercice 2

Créer ListeFruits.jsx qui affiche les fruits Redux dans <ul><li>.

Exercice 3

Ajouter un bouton ❌ pour supprimer un fruit via dispatch.

Exercice 4

Ajouter une étoile ⭐ toggle favori: true/false dans Redux.

Exercice 5

Afficher dans <p> uniquement les fruits favoris, **via un selector global** selectFruitsFavoris.

Exercice 6

Afficher dans <h3> le total des fruits, **calculé dans un selector global**.

Exercice 7

Créer un bouton **Vider le panier** qui reset le tableau Redux.

Exercice 8

Ajouter une propriété quantite (par défaut 1), affichage : Banane x 1.

Exercice 9

Ajouter ➕ ➖ pour modifier quantité (1 à 20 max).

Exercice 10

Champ **recherche** qui filtre l'affichage sans modifier Redux.

Formateur : AARAB HAFID

Exercice 11

Créer un selector global selectFruitsTries qui retourne les fruits triés A → Z.

Exercice 12 (Props dans selector)

Créer un selector avec **paramètre props** :

```
const selectFruitByName = (state, name) =>
  state.fruits.items.find(f => f.nom === name)
```

Afficher dans un composant : "Le fruit Banane existe ? oui/non"

Exercice 13 (useStore + getState)

Dans TestStore.jsx, récupérer store sans useSelector :

```
const store = useStore()
const fruits = store.getState().fruits.items
```

Afficher le résultat dans un console.log.

Exercice 14 (différence sync vs async)

Créer 2 boutons :

- Ajouter sync → ajoute directement un fruit
- Ajouter async → ajoute après 2s + affiche “Ajout en cours…”

Exercice 15 (async avec API locale)

Créer une API locale fruits.json :

```
[
 { "id": 1, "nom": "Pomme", "quantite": 1, "favori": false },
 { "id": 2, "nom": "Banane", "quantite": 1, "favori": true }
]
```

Dans un thunk fetchFruits(), charger les fruits via fetch('/fruits.json') dans Redux avec loading.

Partie 2 : Todo List / Tasks

Exercice 16

AjoutTask.jsx input + bouton pour ajouter une tâche à Redux.

Exercice 17

ListeTasks.jsx affiche les tâches dans <li>.

Exercice 18

Ajouter bouton 🗑 pour supprimer une tâche.

Exercice 19

Toggle tâche terminée → ligne barrée si vrai.

Exercice 20

Afficher les tâches terminées dans un <p> en utilisant un **selector global**.

Exercice 21

Afficher dans <h3> le nombre de tâches **non terminées**, calculé dans un selector.

Exercice 22

Bouton **Supprimer les tâches terminées** (dispatch).

Exercice 23

Champ de recherche qui filtre l'affichage sans toucher Redux.

Exercice 24 (getState dans un async thunk)

Créer addTaskIfEmpty() :

- Lire Redux avec getState()
- Si aucune tâche, en ajouter une automatiquement

Exercice 25 (fetch API locale async tasks)

Créer tasks.json :

```
[
  { "id": 1, "titre": "Apprendre Redux", "done": false },
  { "id": 2, "titre": "Créer un store", "done": true }
]
```

Charger les données avec fetch('/tasks.json') dans un thunk Redux + affichage loading.